VIRL — Launch & UX Enhancement Brief for Claude Code

Repo: `virl-app/VIRL-app` **Primary file:** `index.html` (the main React app, single-file Babel standalone) **Launch date:** May 15, 2026 at 9:00 AM CDT (10:00 AM EDT / 14:00 UTC) **Beta access code:** `LETSGOVIRL`

🎯 Prompt to start (copy this into Claude Code)

> *"Read the file at `docs/virl-launch-brief.md` first. Do not start coding yet. After reading, tell me back: (1) the order you'll tackle the work in, (2) any questions you have, and (3) which task you'll start with. After my approval, work on ONE task at a time. After every file you modify, print a one-line status update. After completing each task, commit with a clear message, push to a new branch, open a PR, and wait for my review before starting the next task. Do not work on multiple tasks in parallel. Do not go silent for more than 5 minutes — if you're thinking, say so."*

📋 Order of operations (work in this exact sequence)

The first 3 are launch-critical. Don't move to enhancements until these are shipped.

🚨 Launch-Critical (must ship before May 15)

1. **Profile Persistence in Supabase** — fixes data loss across devices
2. **Pre-Launch Gating with Beta Code** — hides app until launch day
3. **Waitlist Landing Page** — drives signups before launch

🚀 Conversion & Retention Enhancements (ship after launch)

4. Sample Plan Generator (highest conversion lift)
5. In-Product Referral Modal (drives virality)
6. Profile Completeness Meter (activation)
7. Trial Countdown Banner (conversion urgency)

8. Friendly Credit Counter (quick win)
9. Generic AI Comparison Toast (first-plan magic moment)
10. Vault Smart Reference (retention signal)
11. Rotating Creator Testimonials (in-product social proof)
12. Screenshot Upload for Results Tracking (data unlock)
13. Sunday Results Nudge (retention loop)

⚙️ Critical context for Claude Code

Tech stack:

- Single-file React app via Babel standalone (`<script type="text/babel">`)
- All components use `function` declarations, NOT arrow functions in component bodies
- Supabase for auth + database (URL: `https://xjewyayhtyalhxekphzg.supabase.co`)
- Anthropic Claude API for generation, called via `chat.js` Vercel serverless function
- Stripe for payments
- localStorage key: `virl_v5`

Brand palette — defined in `const B = { ... }` around line 135 of `index.html` . Use `B.navy` , `B.blue` , `B.coral` , `B.coralSoft` , `B.navySoft` , `B.green` , `B.greenSoft` , `B.amber` . NEVER hardcode hex values.

Fonts: `'Italiana', serif` for display, `'Jost', sans-serif` for body. Already loaded globally.

Critical Babel rules (these will break the build if violated):

- ❌ No optional chaining `?.` in JSX
- ❌ No template literals in JSX
- ❌ No em-dashes in raw JSX text — use en-dash `–` or word
- ❌ No `static` keyword in classes
- ❌ Don't put `}} style={{` on the same line — split across lines

Existing helpers to reuse (do not rewrite):

- `callAPI(prompt, maxTokens, imageBase64, imageType, cost, token, model, systemPrompt)` — all Claude API calls
- `extractJSON(text)` — robust JSON parsing with truncation recovery
- `buildProfileCtx(profile)` — builds personalized profile context
- `save(obj)` and `JSON.parse(localStorage.getItem("virl_v5")||"{}")` — local storage
- `logEvent(userId, eventName, properties, token)` — analytics events
- `getCredits()`, `spendCredits(amount)`, `getNextMonday()`, `daysUntilReset()` — credit logic
- `fileToBase64(file)`, `extractVideoFrame(file, atSeconds)` — file handling

Definition of done for each task:

1. Code parses cleanly (run sucrase + vm.Script verification)
2. App still loads, auth still works, all existing flows still complete
3. Add `// [TASK N]` comments above any new code blocks
4. Match existing code style — function declarations, `style={Object.assign({},base,{...})}`, no shortcuts
5. Commit with clear message, push branch, open PR, wait for review

---

🚨 TASK 1 — Profile Persistence in Supabase (LAUNCH CRITICAL)

Problem: Right now, the user's Creator Profile is only stored in `localStorage`. This means if they clear browser data, switch devices, or use a different browser, their profile (their voice, audience, off-limits, etc.) is gone — and every output reverts to generic AI. This is the foundation of the entire product.

Goal: Store profile in Supabase keyed on `user_id`. Use localStorage as a fast read-through cache. On login, hydrate from Supabase. On save, write to both.

Implementation:

1. Create the Supabase table (Claude Code: include this SQL in the PR description so the user can run it in Supabase SQL editor):

```
CREATE TABLE IF NOT EXISTS profiles (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  profile JSONB NOT NULL DEFAULT '{}',
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- RLS policies
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users can read own profile"
  ON profiles FOR SELECT
  USING (auth.uid() = user_id);

CREATE POLICY "Users can insert own profile"
  ON profiles FOR INSERT
  WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Users can update own profile"
  ON profiles FOR UPDATE
  USING (auth.uid() = user_id);

-- Auto-update timestamp
CREATE OR REPLACE FUNCTION update_profiles_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER profiles_updated_at
  BEFORE UPDATE ON profiles
  FOR EACH ROW
  EXECUTE FUNCTION update_profiles_updated_at();
```

2. Add helper functions in `index.html` (near the existing Supabase helpers around line 60-120):

```
async function saveProfileToSupabase(profile, userId, accessToken) {
  if (!userId || !accessToken) return false;
  try {
    const res = await fetch(SUPABASE_URL + "/rest/v1/profiles", {
      method: "POST",
      headers: {
        "apikey": SUPABASE_ANON_KEY,
```

```
        "Authorization": "Bearer " + accessToken,
        "Content-Type": "application/json",
        "Prefer": "resolution=merge-duplicates"
      },
      body: JSON.stringify({ user_id: userId, profile: profile })
    });
    return res.ok;
  } catch(e) { return false; }
}

async function loadProfileFromSupabase(userId, accessToken) {
  if (!userId || !accessToken) return null;
  try {
    const res = await fetch(SUPABASE_URL + "/rest/v1/profiles?user_id=eq." + user
      headers: {
        "apikey": SUPABASE_ANON_KEY,
        "Authorization": "Bearer " + accessToken
      }
    });
    if (!res.ok) return null;
    const data = await res.json();
    return (data && data[0] && data[0].profile) || null;
  } catch(e) { return null; }
}
```

3. Wire the cache pattern:

- **On login** (in the `onAuth` callback or wherever `setSession` is called): Call `loadProfileFromSupabase()`. If a profile exists, set it via `setProfile()` AND save to localStorage. If no profile in Supabase but one in localStorage, push the local one to Supabase (one-time migration for existing users).
- **On profile save** (in `CreatorProfile` component's save handler around line 1270): After saving to localStorage, also call `saveProfileToSupabase()`. Don't block the UI — fire and forget with a quiet console log on failure.
- **On logout:** Clear localStorage profile so the next user doesn't see it.

4. Add a small status indicator in the profile screen: a tiny "Synced ✓" or "Syncing..." text near the save button so users see their data is safe.

Acceptance test: Log in on Browser A, fill in profile, save. Log in on Browser B as same user → profile should appear automatically.

🚨 TASK 2 — Pre-Launch Gating with Beta Code (LAUNCH CRITICAL)

Problem: Both the landing page and the app are currently public. Need to hide them until May 15, 2026 at 9 AM CDT, while still allowing beta testers in.

Goal: Build a clean gate that accepts beta code `LETSGOVIRL` to bypass, and auto-disables itself when the launch date passes.

Implementation:

1. Create a reusable gating mechanism that wraps both the landing page (`landing.html` or wherever it lives) and the app (`index.html`).

2. Logic (place at the top of each gated page, before the main content renders):

```
const LAUNCH_TIMESTAMP = new Date('2026-05-15T09:00:00-05:00').getTime();
// 9:00 AM CDT = UTC-5 in May (CDT is daylight savings)

const BETA_CODE = 'LETSGOVIRL';

function checkAccess() {
  // 1. Past launch date? Allow everyone in.
  if (Date.now() >= LAUNCH_TIMESTAMP) return true;
  // 2. Beta code stored in localStorage from previous visit? Allow.
  if (localStorage.getItem('virl_beta_access') === '1') return true;
  // 3. URL param ?beta=LETSGOVIRL? Store it and allow.
  const url = new URL(window.location.href);
  const beta = url.searchParams.get('beta');
  if (beta === BETA_CODE) {
    localStorage.setItem('virl_beta_access', '1');
    // Clean URL
    url.searchParams.delete('beta');
    window.history.replaceState({}, '', url.toString());
    return true;
  }
  return false;
}
```

3. The gate screen that shows when access is denied — should match VIRL aesthetic (cream/peach palette from the brighter waitlist page):

- Centered card with VIRL logo + “Finally, a strategy that sounds like you.” tagline
- Headline: “Launching May 15 at 9 AM CDT.”

- Subhead: “Be the first to know when we go live.”
- Two paths:
  - **Primary CTA:** “Join the waitlist” → links to the waitlist page
  - **Secondary:** Small “Have a beta code?” expandable input. Submitting `LETSGOVIRL` stores it in localStorage and reloads the page.
- Below: a live countdown to launch (days/hours/mins/secs)
- No nav, no footer links other than legal

4. Auto-deactivation: Once `Date.now() >= LAUNCH_TIMESTAMP`, the gate stops rendering automatically. No manual intervention needed on launch day. Test by temporarily setting LAUNCH_TIMESTAMP to a past time to confirm it lets you through.

5. Beta tester experience: Anyone you give a URL like `https://app.govirl.ai?beta=LETSGOVIRL` will be auto-admitted on first visit. The code is stored locally so subsequent visits work even without the URL param.

Acceptance test:

- Visit app without beta code → see gate screen with countdown
- Visit `?beta=LETSGOVIRL` → granted access, URL cleans up, can use app normally
- Subsequent visits without param → still admitted (localStorage)
- Manually advance system clock past May 15 → gate disappears, app loads normally for everyone

---

🚨 TASK 3 — Waitlist Landing Page (LAUNCH CRITICAL)

Problem: Need a polished, conversion-focused waitlist page to drive signups in the weeks before launch.

Reference file: I’ve already designed this page — full HTML at the path the user provides. Match this design exactly. Aesthetic notes:

- Cream/peach base (`#FFF9F4`), NOT dark
- Bright pastel section backgrounds (pink, sky, mint, lemon)
- Coral→peach gradient CTAs
- Italiana display font + Jost body font

- Animated floating background blobs (peach + sky)
- Sunrise gradient "founding member" perk card
- Live countdown to May 15, 2026 at 9 AM CDT
- Animated VIRL plan preview card with 3 fading-in plan cards (use the structure from the existing app's Plan tab — show example posts with platform badges, "why" reasoning)
- 4 colorful stat tiles ("60s · 7 platforms · 100% your voice · $60 saved")
- 6 feature cards with colored top borders
- 3-step "How it works"
- FAQ accordion

The "wow factor" requirement: The animated plan preview is critical. It should look like the actual VIRL Plan output — same card style, same colored day badges, same "Why:" reasoning lines. Use staggered fade-in animations (0.2s, 0.6s, 1.0s delays) so users see the plan being "generated" in real time. This is what sells the product before they can use it.

Founding member offer (UPDATED):

- First **100** users (not 500) get $5 off per month for first year
- Two payment options shown:
  - **Monthly:** $20/mo for 12 months (save $60), then standard $25/mo
  - **Annual:** $225 first year (save $75 — that's 3 months free), then standard $250/yr
- Show both options side-by-side in the perk card

Waitlist signup flow:

- Email field + "Save my spot" button
- On submit, POST to `/api/waitlist` (the endpoint Claude Code already built — Task 0 already complete)
- On success, swap form for confirmation card: "You're in. ✦ Check your email May 15 for your founding member code."
- Live "spots claimed" counter pulled from Supabase: "X of 100 spots claimed" with progress bar

Supabase table for waitlist (Claude Code: include this SQL in the PR description):

```
CREATE TABLE IF NOT EXISTS waitlist (
```

```
  email TEXT PRIMARY KEY,
  source TEXT,
  founding_code TEXT,
  notified BOOLEAN DEFAULT false,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

ALTER TABLE waitlist ENABLE ROW LEVEL SECURITY;

-- Anonymous users can insert (signup) but not read
CREATE POLICY "Anyone can sign up"
  ON waitlist FOR INSERT
  WITH CHECK (true);
```

Founding member code generation:

- When a user signs up AND they're in the first 100, auto-generate a unique code like `VIRL-FOUNDER-XXXX` (4 random alphanumeric chars). Store in `founding_code` column.
- On May 15 launch day, the user will need a script to send launch emails with these codes (separate task — flag for later).
- For now, just store the code so it's ready.

Routing:

- The waitlist page should be accessible at `/waitlist` AND should be the page the access gate links to from the gated landing page/app.
- When the gate auto-deactivates on May 15, the waitlist page can stay live but should add a banner at top: "We've launched! [Sign up for VIRL →]" linking to the app signup flow.

Acceptance test:

- Visit `/waitlist` → see polished page with countdown, animated plan preview, signup form
- Submit email → success state shows, email lands in Supabase `waitlist` table with auto-generated `founding_code`
- Spots counter increments correctly
- Page works on mobile (responsive)

🚀 TASK 4 — Sample Plan Generator

Goal: Show new users the magic in 30 seconds instead of 5 minutes. Highest conversion lift.

Where: In the plan generator screen (inside `App()` function, look for `tab === "plan"` block around line 2470-2570).

Spec:

- If user has never generated a plan (`localStorage.getItem('virl_has_generated') !== '1'`), show prominent secondary CTA above the form: **"Skip the form — show me a sample plan first"**
- Clicking calls existing `generate()` function with profile-derived defaults: `profile.myPlatforms` (or `["TikTok","Instagram"]`), `profile.niche` (or `"Lifestyle"`), goal `"Grow followers"` , formats `["Reels","Posts"]` , follower tier `"1K-10K"`
- After first successful generation, set `localStorage.setItem('virl_has_generated','1')` so sample button hides and full form shows
- Cost: still uses 3 credits — don't give free plans, just reduce friction

Visual: Outlined button with `B.coral` border + coral text on white, full-width, above the existing form. Small text under it: "Uses 3 credits · You have 20 this week · Customize after"

🚀 TASK 5 — In-Product Referral Modal

Goal: Drive viral coefficient. Trigger at moments of success.

Spec:

- Add localStorage counter `virl_plans_completed` (increment after each successful plan generation)
- Show modal when counter hits **3, 7, and 15** (not every time)
- Copy: **"You've built 3 plans this week 🎉 Share VIRL with a creator friend and you both get 2 weeks free."**
- Modal contents:
  - Copy button for referral link: `https://app.govirl.ai?ref=`

`{firstEightCharsOfUserId}`

  - Share buttons for X, Instagram Stories, copy to clipboard
  - "Maybe later" dismiss link
- Use `localStorage` flag `virl_referral_shown_N` per milestone (each shows only once)
- Modal is dismissible, doesn't block UI
- Add `// TODO: backend referral credit on signup` comment where the backend hook would go

🚀 TASK 6 — Profile Completeness Meter

Goal: Gamify profile setup, signal which fields matter.

Where: Top of `CreatorProfile` component (around line 1270).

Spec:

- Calculate completeness from `profile` object:
  - **Basic (33%):** `name` + `audience`
  - **Dialed (66%):** + `voice` + at least 1 `vibe` + `knownFor`
  - **Elite (100%):** + `offLimits` + `inspiration` + `sampleCaption`
- Horizontal progress bar with tier label
- Colors: Basic = `B.amber`, Dialed = `B.blue`, Elite = `B.green`
- Tier-specific copy next to bar:
  - Basic: "VIRL has the foundation. Add voice to make plans sound like you."
  - Dialed: "Plans will sound like you. Add off-limits to make them precise."
  - Elite: "Your plans are fully personalized. ✦"
- Small badge next to each field label showing what tier it unlocks (`+Basic` , `+Dialed` , `+Elite`) in muted text

---

🚀 TASK 7 — Trial Countdown Banner

Goal: Layered urgency in days 11–14 of trial.

Where: Top of main app view inside `App()`, above tab navigation. Render conditionally.

Spec:

- Compute `daysIntoTrial` from `signupDate`
- Show banner when `daysIntoTrial >= 11 && daysIntoTrial < 14 && plan === 'trial'`
- Copy varies by day:
  - Day 11: "3 days left in your free trial"
  - Day 12: "2 days left – lock in 20% off your first year"
  - Day 13: "Last day of trial – continue for $20/mo (save $60 year one)"
- Background: `B.coralSoft`, text: `B.coral`, bold day count
- Close (×) button — dismissed banner reappears next session, hidden current session via `sessionStorage`
- "Upgrade now" button on right triggers existing upgrade modal

---

🚀 TASK 8 — Friendly Credit Counter

Goal: Users understand value, not tokens.

Where: Wherever credits are displayed (likely around lines 2160–2180).

Spec:

- Next to credit number, add small secondary line translating credits using `CREDIT_COSTS = {plan:3, script:2, caption:1, scan:2, regen:1}`:
  - Plans available = `Math.floor(credits / 3)`
  - Captions available = `Math.floor(credits / 1)` (cap display at 20)
  - Scans available = `Math.floor(credits / 2)`
- Display: `"~{plans} plans, {captions} captions, {scans} scans"` in `B.muted` at 11px
- Only show when credits between 3 and 150

🚀 TASK 9 — Generic AI Comparison Toast

Goal: The “wow, VIRL really sounds like me” moment.

Spec:

- After plan generation completes for user’s first plan ever (`localStorage.getItem('virl_first_plan_shown') !== '1'`), show dismissible comparison card above the plan cards
- Card contents:
  - Eyebrow: “SEE THE DIFFERENCE”
  - Headline: “Here’s what a generic AI would have written.”
  - Side-by-side (stack on mobile):
    - Left (grayed): “Generic AI” — hardcoded caption like `"Loving this morning routine ✨ Start your day right! 🌅 #morningvibes #selfcare #inspiration"`
    - Right (branded): “VIRL” — first caption from generated plan
  - Below: “Every post in your plan is written like this — built on your voice, audience, and goals.”
  - “Save this image” or “Copy comparison” button
- After dismissal, set `localStorage.setItem('virl_first_plan_shown','1')` — never shows again
- Use `B.coral` accent on VIRL side, `B.muted` low-opacity on Generic AI side

---

🚀 TASK 10 — Vault Smart Reference

Goal: Make users feel VIRL is learning from them.

Spec:

- Before calling Claude API with `planPrompt` , inspect `vault` array
- Extract patterns: which platforms appear most, which formats, recurring themes in `title` fields
- Add line to `planSystemPrompt` : `"The creator has saved {N} posts they love. They`

```
lean toward {top platform} and {top format}. Weight similar styles higher in
this week's plan."
```

- In UI above plan results, show banner when vault has 3+ items: **"Based on your Vault, VIRL is weighing {format} content higher this week."**
- Banner style: `B.navySoft` background, `B.navy` text, small Italiana serif heading

---

🚀 TASK 11 — Rotating Creator Testimonials

Goal: In-product social proof at decision moments.

Where: (1) Inside upgrade modal (around line 2280). (2) On Results tab empty state (around line 1553).

Spec:

```
const TESTIMONIALS = [
  {name:"Maya R.", handle:"lifestyle creator · 28K", quote:"I cancelled three oth
  {name:"Jordan K.", handle:"business creator · 12K", quote:"Sunday planning went
  {name:"Sam T.", handle:"wellness creator · 41K", quote:"It picked up on my voic
  // PLACEHOLDER — replace with real beta user quotes post-launch
];
```

- Create `<Testimonial />` component that picks one at random on mount
- Style: quote in Italiana italic, name/handle in Jost 12px muted, 5-star rating above (★★★★★ in `B.amber`)
- Drop into upgrade modal header area + Results empty state

🚀 TASK 12 — Screenshot Upload for Results Tracking

Goal: Remove friction from results logging using Claude vision.

Where: On `PlanCard` component (line 752), specifically the `showLog` state block.

Spec:

- Add second path: **"Paste screenshot instead"**
- File input accepting images

- On upload:
  - Convert to base64 with existing `fileToBase64`
  - Call: `callAPI(prompt, 500, base64, mimeType, 0, token, "claude-haiku-4-5-20251001", systemPrompt)`
  - System prompt: `"You are a data extraction assistant. The user uploaded a screenshot of social media analytics. Extract view count, like count, and save/comment count. Return ONLY JSON: {\"views\": number, \"likes\": number, \"saves\": number}. Return null for unclear fields."`
  - Parse with `extractJSON`
  - Pre-fill input fields with extracted numbers
  - User reviews and saves
- Cost: 0 credits (retention feature)
- Fallback: if extraction fails, "Couldn't read screenshot — tap to enter manually"

🚀 TASK 13 — Sunday Results Nudge

Goal: Batched behavior beats per-post behavior.

Spec:

- On Results tab, check `new Date().getDay() === 0` (Sunday)
- If Sunday AND user has plans AND any plan has no results logged, show dismissible banner
- Copy: **"How did your posts perform this week? Log results in 90 seconds — it helps VIRL learn what's working for you."**
- "Log all at once" button pops modal showing each unlogged plan with quick-entry fields
- Use `sessionStorage` to dismiss for current session
- Style: `B.navySoft` background, `B.navy` text, Italiana heading

📁 Files that may need changes

- **Primary:** `index.html` (main app) — most tasks
- **New file:** `waitlist.html` (landing page)
- **New file or update:** `landing.html` (gated)
- **New file:** `api/waitlist.js` — already exists, may need updates for founding code generation
- **Secondary:** `chat.js` (Vercel function) — may need tweaks for screenshot extraction routing
- **Documentation:** SQL migrations for new Supabase tables (include in PR descriptions)

🛡️ Safety habits — please follow

1. **One task at a time.** Do not start Task 2 until Task 1 is reviewed and merged.
2. **Commit and push after every task.** Open a PR per task with a clear description.
3. **One-line status updates after every file modified.** No silent stretches longer than 5 minutes.
4. **If you get stuck or unsure, ask.** Don't guess and don't loop on broken approaches.
5. **Verify syntax before committing.** Run sucrase + vm.Script check on any modified `index.html`.
6. **Include any required SQL migrations** in the PR description so the user can run them in Supabase before merging.

End of brief. Estimated total time with focused execution: 6-10 hours of work across 13 PRs. Realistic completion: 2-3 days if user is responsive to PR reviews.